

ST2195 Homework Exercise

University of London
Programming for Data Science
Hesham M. Al-Ammal
December 2025

Topic: The Epsilon-Greedy Algorithm & Git Workflow

Submission: Git

Languages Required: Python and R

Deadline: Optional but for feedback is 21 Dec 2025

1 Background

In this project, you will simulate a reinforcement learning foundation problem known as the **Multi-Armed Bandit**.

Imagine you are a Data Scientist trying to optimize click-through rates (CTR) for a website. You have $K = 5$ different versions of an advertisement (referred to as “Arms”), and you need to decide which one to show to users to maximize clicks.

Since you do not know the true conversion rate of each ad, you must balance **Exploration** (trying random ads to gather data) and **Exploitation** (showing the ad that currently looks the best). You will implement the **Epsilon-Greedy** algorithm to solve this.

2 The Algorithm

You need to simulate N users visiting the site. The procedure is as follows:

1. **Setup:** Define $K = 5$ ads. Assign each ad a “True Probability” of being clicked (e.g., $p = [0.1, 0.5, 0.2, 0.05, 0.3]$). *Note: The algorithm does not know these true values; it must learn them.*
2. **Initialization:** Create an array to store the estimated probability of each arm (Q) and the count of times each arm was pulled (N_{arm}). Initialize all values to zero.
3. **Iterate:** For visitor $i = 1, \dots, N$:
 - **Generate Random Number:** Draw $u \sim \mathcal{U}(0, 1)$.
 - **Conditional Decision:**
 - **Explore:** If $u < \epsilon$: Choose a random ad from the K options.
 - **Exploit:** Else: Choose the ad with the highest current estimated probability in Q ($\text{argmax}(Q)$).
 - **Simulate Reward:** “Pull” the chosen arm. Simulate a click (1 or 0) using a Bernoulli trial based on that arm’s *True Probability*.
 - **Update Estimates:** Update the estimated probability (Q) for the chosen arm using the incremental mean formula:

$$Q_{new} = Q_{old} + \frac{1}{N_{arm}}(Reward - Q_{old})$$

3 Tasks

Part A: Implementation and Visualization

Implement the simulation in both **R** and **Python**.

1. Define a function (or method) `run_bandit(epsilon, n_steps)` that executes the algorithm described above.
2. Set the True Probabilities to $[0.10, 0.50, 0.20, 0.05, 0.30]$.
3. Run the simulation with $N = 10,000$ and $\epsilon = 0.1$.
4. **Visualize:** Plot the **Cumulative Average Reward** over time.
 - The x-axis should be the iteration number (1 to N).
 - The y-axis should be the average reward received up to that point.
 - Add a horizontal line representing the theoretical maximum probability (0.50) to see if your algorithm converges toward the optimal solution.

Part B: Parameter Analysis (The Explore-Exploit Tradeoff)

Just as MCMC depends on step size, the Bandit algorithm depends heavily on ϵ .

1. Run the experiment three separate times with different values: $\epsilon = 0.01$ (Greedy), $\epsilon = 0.1$ (Balanced), and $\epsilon = 0.5$ (Random).
2. **Combined Plot:** Produce a single plot overlaying the **Cumulative Average Reward** curves for all three ϵ values.
3. **Analysis:** In your report, answer the following based on your plot:
 - Which value converges to the optimal solution fastest?
 - Which value achieves the highest long-term stability?
 - Does a high ϵ (0.5) prevent the algorithm from ever reaching the theoretical maximum? Why?

4 Submission Instructions (Git Workflow)

This assignment tests your ability to use Git for collaboration. You must submit your solution by interacting with the instructor's repository on GitHub (heshaaam). Please submit one Jupyter notebook and one R Markdown. Include your student ID and name in the first markdown cell of the notebooks.

Prerequisites: Ensure you have a GitHub account and Git installed locally.

Step-by-Step Submission

1. Clone the Repository

Open your terminal and clone the instructor's repository:

```
git clone https://github.com/heshaaam/st2195-bandit-assignment.git
cd st2195-bandit-assignment
```

2. Create Your Branch

Do *not* work directly on the `main` branch. Create a branch using your student ID:

```
git checkout -b student_ID_bandit
```

3. Organize Your Files

Create a directory named after your student ID. Inside, create two sub-folders: **Python** and **R**.

- Save your Python Jupyter notebook as `bandit_solution.ipynb` inside the **Python** folder.
- Save your R Markdown file as `bandit_solution.Rmd` inside the **R** folder.

4. Commit Your Work

Stage and commit your files with a clear message:

```
git add .
git commit -m "feat: Implemented Epsilon-Greedy algorithm in R and Python"
```

5. Push and Submit

Push your branch to the remote repository:

```
git push origin student_ID_bandit
```

Appendix: Skill Mapping

This table demonstrates how this Bandit problem tests the same skills as the original MCMC coursework.

Skill	Original MCMC (Part 1)	New Bandit Problem
Randomness	Sample Normal/Uniform for proposals	Sample Uniform for ϵ ; Bernoulli for Reward
Loops	Iterate N times for chain generation	Iterate N times for user visits
Conditionals	if $u < \text{ratio}$ (Accept/Reject)	if $u < \text{epsilon}$ (Explore/Exploit)
Data Storage	Store sequence of x values	Store history of rewards and Q-values
Functions	Define PDF $f(x)$	Define Environment and Agent logic
Visualization	Convergence of density (Hist vs Curve)	Convergence of reward (Trajectory vs Optimal)
Jupyter and R notebook	Skills in creating and editing notebooks with code, notes, and visualization	same
Diagnostics	$\hat{R}$ across grid of s	Reward curves across grid of ϵ